COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

PACKAGE 07 / VOLUME 06

Laboratory Workbook

20 full labs • Five integrating projects

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Laboratory Workbook 4

Common commands 4

Common evidence record 4

Module 26: Collision detection and physical simulation 5

Lab 26.1: Broad phase, narrow phase and geometric robustness 5

Lab 26.2: Rigid bodies, angular motion and collision impulses 7

Lab 26.3: Constraints, iterative solvers and continuous collision detection 9

Lab 26.4: Physics testing, determinism and engine integration 11

Module 26 project 12

Module review evidence 13

Module 27: Animation, audio and interactive timing 14

Lab 27.1: Skeletons, transforms and skinning 14

Lab 27.2: Animation blending, state machines and inverse kinematics 16

Lab 27.3: Digital audio, mixing and real-time callback discipline 18

Lab 27.4: Spatial audio, animation events and multimodal accessibility 20

Module 27 project 21

Module review evidence 22

Module 28: Game AI, decision systems and machine learning 23

Lab 28.1: Navigation graphs, steering and dynamic obstacles 23

Lab 28.2: Behavior trees, utility systems and explainable decisions 25

Lab 28.3: Adversarial search, planning and simulation budgets . . . 27

Lab 28.4: Learning systems, neural inference and responsible evaluation . . . 29

Module 28 project . . . 30

Module review evidence . . . 31

Module 29: Multiplayer simulation and online services 32

Lab 29.1: Authority, prediction and reconciliation . . . 32

Lab 29.2: Snapshots, interpolation, relevance and lag compensation . . . 34

Lab 29.3: Lockstep, rollback and deterministic replay . . . 36

Lab 29.4: Match services, identity, persistence and failure recovery . . . 38

Module 29 project . . . 39

Module review evidence . . . 40

Module 30: Production quality and the undergraduate engine capstone 41

Lab 30.1: Profiling, frame budgets and performance regression tests . . . 41

Lab 30.2: Compatibility, accessibility, localization and human testing . . . 43

Lab 30.3: Packaging, patching, modding and supply-chain boundaries . . . 45

Lab 30.4: Capstone review, release and technical postmortem . . . 47

Module 30 project . . . 48

Module review evidence . . . 49

Gate F handoff 50

Laboratory Workbook

Each lesson has two levels of implementation: a small solve(payload) kernel with a matching demonstration/reference and a full subsystem laboratory. The latter retains the original practical assignment and adds procedure, adversarial cases and evidence. The kernel is not the complete lab. Native engine transfer, actual audio/device stress, maintained-library comparison, process/socket networking and human observations are separately required where stated.

Write predictions before running code. Record actual results and failure causes in templates/LAB_JOURNAL.md. Keep the reference closed until after your independent attempt. Each lab card names evidence, not precompleted student work. Tests exercise a declared teaching contract rather than certifying production security.

Common commands

```
python starter/python/run_tests.py --target references
python starter/python/run_tests.py --target exercises --lesson 26.1
python starter/python/demos/lesson_26_1.py
python starter/python/test_properties.py
```

Replace the lesson ID/file suffix for another lesson. The 20 exercises intentionally raise NotImplementedError until implemented; normal testing reports that as failure. Do not import reference code into an exercise to obtain a false pass.

Common evidence record

Record lesson ID, date, hypothesis, input/seed, environment, command, expected result, actual result, failing case, correction and transfer evidence. Separate synthetic inputs from measured performance. For numerical tests, state units and tolerance; for timing, state start/end boundaries and workload; for human tests, state the task and observed outcome.

Module 26: Collision detection and physical simulation

Lab 26.1: Broad phase, narrow phase and geometric robustness

Original assignment and setup

Implement a grid broad phase and circle or box narrow phase. Pass: no true pairs are omitted compared with brute force over randomized scenes, and boundary-touching behavior is documented.

Use a 2D world measured in meters. Start with circles, then represent one static wall with an AABB. Keep stable integer body IDs and a documented closed-set touching policy. The starter kernel supplies grid candidate generation and circle contacts only; normals, manifolds, masks and engine ownership remain your work.

Exercise kernel contract

Input: circles as [integer_id, x, y, radius] and positive cell_size. IDs are unique; radii are positive. At most 512 circles and 4096 occupied cells per circle. Return sorted canonical candidate pairs and sorted true contact pairs. Closed-set touching counts as contact. Reject non-finite data and excessive cell coverage before populating the grid.

Starter: `starter/python/exercises/lesson_26_1.py`

Demonstration: `starter/python/demos/lesson_26_1.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 26.1`

Procedure

1. On paper, enumerate the three pairs for three circles, identify exact tangencies, and draw every covered cell. Include a negative coordinate and one radius larger than the cell width.
2. Implement the kernel without importing the reference. Retain an independently written O(n^2) narrow-phase oracle; never make the oracle call your grid or reuse its membership logic.
3. Run 200 seeded scenes with at least 30 circles each. Check true_contacts is a subset of candidates, then check that the final contacts equal the oracle. Save a complete failing scene before stopping.
4. Extend contacts with depth and a normal pointing A to B. Define the coincident-center fallback. Add collision layers and verify that filtering is performed identically in the oracle and engine path.
5. Time uniform, clustered and giant-object scenes separately. Record grid construction, pair generation and narrow-phase work rather than only one total. Transfer the index into the

fixed-step simulation, independent of camera visibility.

Adversarial and boundary cases

Exact cell boundary; negative coordinate; coincident centers; duplicate IDs; a shape covering too many cells; spawn/despawn during the scene; non-finite position rejected before indexing.

Required evidence

A boundary sketch, seed list, candidate/contact counts, first-failure capture format, three workload tables and an engine trace showing off-camera contacts.

Pass conditions

No omitted true contacts across the declared test suite; all invalid inputs fail before unsafe indexing; normal direction and touching policy are explicit. Performance gains are not required when measurements show the grid is unsuitable.

Native engine transfer

Port the broad-phase ownership and pair-order contract into the Volume 5 engine. Keep the brute-force oracle in a debug/headless target, independent of the grid implementation.

Extension after the required pass

Compare a maintained dynamic AABB tree with the grid for heterogeneous object sizes. State setup cost and distribution, not just the name of the faster data structure.

Lab 26.2: Rigid bodies, angular motion and collision impulses

Original assignment and setup

Implement center-of-mass impulses and a simple rotational case. Pass: momentum is conserved in isolated tests within tolerance, static bodies remain fixed, and energy behavior matches the chosen restitution model.

Use two isolated dynamic bodies with no gravity, damping or constraints for conservation tests. Then add a static wall and a contact offset from the center of mass. The 1D kernel does not implement angular dynamics.

Exercise kernel contract

Input: va, vb, inverse masses wa, wb, and restitution e in [0,1]. Normal points from A to B along +x. Return va, vb and scalar impulse. Separating contacts and static-static pairs receive zero impulse. Inverse masses must be nonnegative; zero inverse mass is a static body.

Starter: `starter/python/exercises/lesson_26_2.py`

Demonstration: `starter/python/demos/lesson_26_2.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 26.2`

Procedure

1. Derive J = -(1+e)*v_n/K from relative velocity at the contact. Write the inverse-mass and inverse-inertia terms of K, including the two squared cross-product terms in 2D.
2. For two 2 kg bodies with initial velocities 3 and 0 m/s and restitution 0.5, predict J=4.5 N s, final velocities 0.75 and 2.25 m/s, and final kinetic energy 5.625 J. Run the reference only after recording the prediction.
3. Implement the center-of-mass kernel. Sweep restitution 0, 0.5 and 1. Measure linear momentum and kinetic energy before/after. A separating contact must not receive another bounce impulse.
4. Add angular state and an offset contact. Compute contact-point relative velocity, not just center velocity. Reproduce the off-center case in physics_lab.py and explain why its impulse differs from a center-only approximation.
5. Test static-static and dynamic-static pairs without assigning a large finite mass to a wall. Integrate velocities with a fixed timestep, and distinguish solver impulse tests from free-flight integration error.

Adversarial and boundary cases

Zero inverse mass on both bodies; separating velocities; restitution outside [0,1]; contact through center versus offset; reversed normal with swapped body order; highly unequal masses.

Required evidence

A derivation with units, conservation table, off-center free-body sketch, expected angular signs and automated static-body checks.

Pass conditions

Isolated elastic cases conserve momentum and energy within a declared tolerance; inelastic cases conserve momentum but lose the expected energy; static bodies remain fixed and angular signs match the contact convention.

Native engine transfer

Implement separate force and impulse commands in your engine. Add tests proving that the renderer cannot directly overwrite the physics-owned transform of a dynamic body.

Extension after the required pass

Add Coulomb friction as a clamped tangential impulse and report how the friction bound relates to the accumulated normal impulse. Do not claim a complete resting-contact solver.

Lab 26.3: Constraints, iterative solvers and continuous collision detection

Original assignment and setup

Create a fast-projectile tunneling test and a short chain of constraints. Pass: a continuous or swept test catches the projectile, and a plot or table shows constraint error versus iteration count and timestep.

Use a thin wall, one fast projectile and a short constrained chain. Keep a separate table for timestep/substep changes and solver-iteration changes. physics_lab.py supplies one positional-projection illustration, not a production constraint solver.

Exercise kernel contract

Input x, velocity v, slab lo<=hi, and dt>=0. Motion is linear; slab is static and closed. Return earliest time in [0,dt], including 0 for initial overlap, or null for no hit. No rotation or multi-impact solver is implied.

Starter: `starter/python/exercises/lesson_26_3.py`

Demonstration: `starter/python/demos/lesson_26_3.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 26.3`

Procedure

1. Place a wall from x=1.2 to 1.3 m and launch a point at 150 m/s from x=0. With h=1/60 s, show why endpoints alone miss it and derive entry time 0.008 s.
2. Implement the swept slab kernel, including zero velocity, negative velocity and initial overlap. Inflate the wall by projectile radius for the sphere/circle-center approximation and derive the new entry time.
3. Create three distance constraints with one anchored endpoint. Define residual as the maximum absolute length error. Record residual after 1, 2, 4, 8, 16 and 32 iterations.
4. Run an integration experiment at h=1/30, 1/60 and 1/120 s, with the same physical duration and input forces. Report residual, motion and runtime. Do not label a positional-only projection table as a timestep experiment.
5. Handle impact time explicitly: advance to impact, resolve contact, then manage remaining time with a bounded number of impacts or a safe documented fallback. Count the cap being reached instead of silently looping forever.

Adversarial and boundary cases

Projectile beginning inside the wall; tangency; zero-speed axis; two impacts within one step; chain initially folded; unreachable solver tolerance; iteration/impact cap exhausted.

Required evidence

A discrete-versus-swept trace, impact-time derivation, separate iteration and timestep tables, residual definition, runtime and a screenshot or textual state trace of the fast-projectile case.

Pass conditions

Swept detection catches the declared projectile case; initial-overlap policy is explicit; the constraint report separates iteration count from timestep and shows remaining error. Exhausting a budget has a tested finite outcome.

Native engine transfer

Add a debug CCD flag, a per-tick impact cap and residual telemetry to your engine. Demonstrate that a dense contact scene cannot make one tick loop without a bound.

Extension after the required pass

Compare warm-started sequential impulses against cold-started iterations on persistent contacts, keeping total work and contact ordering visible.

Lab 26.4: Physics testing, determinism and engine integration

Original assignment and setup

Build a regression suite with replay hashes on one pinned environment, finite-value checks and analytic tests. Pass: failures identify the first divergent tick and reports distinguish same-machine repeatability from cross-platform determinism.

Freeze the build, platform, timestep, input tape, scene construction order and random seeds. Begin with same-process replay, then a fresh process. Cross-platform determinism is a distinct optional experiment, not inferred from these passes.

Exercise kernel contract

Input left and right lists of JSON-compatible state dictionaries. Return the zero-based first differing state index, or null when equal. Different lengths diverge at the first missing state. Canonicalize with sorted string keys and compact JSON, reject non-finite floats, cap trace length at 10000. This is a Python-environment encoding, not a cross-language wire specification.

Starter: `starter/python/exercises/lesson_26_4.py`

Demonstration: `starter/python/demos/lesson_26_4.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 26.4`

Procedure

1. Define a canonical state schema containing every authoritative value: positions, velocities, sleep flags when used, RNG state and stable entity identity. Exclude pointer addresses, UI state and uninitialized padding.
2. Implement canonical serialization and per-tick hashes. Store the unhashed state around any mismatch so a hash identifies the tick and a structured diff identifies the field.
3. Run a fixed input tape twice. Deliberately alter one input and one hidden state field in separate runs. Confirm that the first-divergent-tick report detects both, rather than comparing only final positions.
4. Use analytic free fall to measure integration error versus timestep. Add finite-value checks and a conservation case. Tolerance-based physical accuracy and bitwise replay are different tests.
5. Build one maintained-library comparison scene, pin its exact version and settings, and record differing collision skins, restitution thresholds, sleep policies and stepping conventions. The package does not bundle or claim to have run that external comparison.

Adversarial and boundary cases

NaN in state; changed construction order; omitted RNG state; tick tape truncated; stale entity generation; altered compiler options; comparison of floats that serialize as integers in one schema but decimals in another.

Required evidence

Environment manifest, canonical schema, input tape, first-divergence report, analytic-error table and a maintained-library comparison with exact build/settings.

Pass conditions

Repeats agree under the declared environment; a deliberately altered state is detected at its first affected tick; finite-value failures stop clearly; the report does not promote one-platform repeatability to untested platform guarantees.

Native engine transfer

Publish a versioned engine state schema and first-divergence replay report. Classify the result as same-environment repeatability unless other named platform combinations were actually tested.

Extension after the required pass

Run the same test on another compiler/architecture and classify each mismatch. Use the maintained library’s documented determinism contract rather than assuming all physics libraries behave identically.

Module 26 project

Build a Physics Evidence Bench. Select a bounded scene containing two isolated colliding bodies, a static wall, one offset contact, a fast projectile and a short constrained chain. Keep scenes small enough to inspect every state transition. Use stable IDs, meters/seconds and a fixed update schedule. A graphical view is useful, but the headless state trace is the regression authority.

Integrate all four lessons: a conservative broad phase, narrow-phase contact convention, center/angular response, swept collision case and replay harness. Record the free-flight integrator separately from the contact solver. Run at least three timestep settings and several solver budgets without hiding energy injection or unresolved penetration. A limitation is acceptable when it is measured and bounded.

Compare at least one matched scene with a maintained library such as Box2D, using the exact library revision and settings. Do not substitute an online claim for running the comparison. The reference package includes a comparison template but does not download or execute that external library. No equality is promised when algorithms, skin widths or restitution thresholds differ.

Deliver scene files, seed/input tape, canonical state schema, invariant tests, first-divergence capture, raw measurements and a comparison report. Review asks: which invariants should hold here, what tolerances are numerical rather than geometric, and which differences are modeling choices?

Module review evidence

Revisit the five review questions in the handbook after completing the integrating project. Use a changed scene or input to demonstrate transfer, not just a repeated definition. Attach your module project record before proceeding.

Module 27: Animation, audio and interactive timing

Lab 27.1: Skeletons, transforms and skinning

Original assignment and setup

Animate a two-joint mesh or line skeleton. Pass: the bind pose reproduces the source geometry, weights are validated and a parent rotation moves descendants consistently.

Use a root plus two articulated links and a small weighted mesh or line skeleton. The kernel covers planar hierarchy composition. Skinning and inverse bind matrices are additional required work.

Exercise kernel contract

Input parents (root=-1) and local transforms [x,y,angle_radians], at most 256 joints. Return world transforms in original joint-index order. Imported order may be arbitrary. Reject cycles, invalid parents, non-finite transforms and mismatched counts. Column-vector parent*local convention; no scale/skinning in the kernel.

Starter: `starter/python/exercises/lesson_27_1.py`

Demonstration: `starter/python/demos/lesson_27_1.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 27.1`

Procedure

1. Write the spaces on every transform: local-to-parent, joint-to-model, inverse-bind and model-to-world. Choose column-vector or row-vector convention and one multiplication order.
2. Implement the planar kernel, including parents that appear after children in storage. Detect cycles and invalid parent indices instead of recursing without a bound.
3. Build a bind pose. Show that G_current * inverse(G_bind) is identity for each joint at bind time, then include the mesh-node correction needed by your asset convention.
4. Assign weights summing to one and validate joint indices. Test a vertex fully assigned to one joint and a vertex blended between two joints. Record whether invalid sums are rejected or normalized.
5. Render at two presentation rates while the simulation/animation samples remain fixed. Rotate the root by 90 degrees and verify descendant positions on paper and in the viewer. Test a non-identity mesh/world transform to expose space mixing.

Adversarial and boundary cases

Hierarchy cycle; disconnected roots; missing joint; weight sum zero; negative weight; non-identity mesh transform; quaternion q versus -q when adding 3D rotation.

Required evidence

A transform-space diagram, bind-pose numeric comparison, weighted-vertex calculation, hierarchy validation tests and articulated playback evidence.

Pass conditions

Bind pose reproduces source geometry within tolerance; parent motion moves descendants consistently; bad weights/joint references are handled intentionally; transform conventions are explicit.

Native engine transfer

Add an immutable skeleton resource and a per-instance pose buffer to the existing engine. Demonstrate two actors sharing one skeleton without sharing playback state.

Extension after the required pass

Add shortest-path quaternion interpolation and a three-joint 3D skeleton. Keep scale/shear limitations explicit when decomposing transforms.

Lab 27.2: Animation blending, state machines and inverse kinematics

Original assignment and setup

Build idle/walk/action transitions and a two-link IK demonstration. Pass: interruptions behave as specified, unreachable targets are handled, and root-motion ownership is documented.

Create idle, walk and action states plus a two-link arm. Keep gameplay state, animation state and the final blended pose separate. The IK kernel handles a planar arm and one elbow branch only.

Exercise kernel contract

Input positive lengths l1,l2 and finite target x,y. Return shoulder, elbow, endpoint and reachable flag. Choose nonnegative elbow angle. Clamp target radius to [abs(l1-l2),l1+l2]; at zero target direction use +x. Equality is reachable; no joint-angle limits or 3D model.

Starter: `starter/python/exercises/lesson_27_2.py`

Demonstration: `starter/python/demos/lesson_27_2.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 27.2`

Procedure

1. Specify entry/exit conditions and interruption rules, including what happens when the action is requested twice or movement begins mid-action. Draw the state-transition table before implementing it.
2. Implement pose blending with a documented time base. Test a transition at its first sample, halfway and completion; verify whether events are emitted by state transitions or animation markers.
3. Derive the cosine-law IK solution. Reconstruct the endpoint with forward kinematics. Clamp the target to the reachable annulus and choose a deterministic direction at zero distance.
4. Decide who owns root motion: either animation proposes displacement that simulation validates, or simulation owns motion and animation follows. Never apply the same displacement in both systems.
5. Run a transition during collision, an unreachable target, and a cancellation while a resource is held. Record cleanup and blend behavior. Add separate visual pose interpolation between simulation ticks.

Adversarial and boundary cases

Inner and outer reach limits; equal links at zero target distance; frame-rate changes during a blend; action canceled on its last frame; duplicate root motion; nearly antipodal quaternions in the 3D extension.

Required evidence

State table, transition traces, IK derivation/reconstruction errors, unreachable-target policy and root-motion ownership decision.

Pass conditions

All declared transitions and interruptions behave as specified; endpoint error is bounded for reachable targets; unreachable targets are finite and explained; root motion has one owner.

Native engine transfer

Implement the state/pose separation in your engine and make the animation trace visible in the inspector. Add one cancellation test that frees an action-owned resource.

Extension after the required pass

Add a joint-angle limit and show why independently clamping solved angles can spoil endpoint accuracy. Compare that simple policy with an iterative constrained solve.

Lab 27.3: Digital audio, mixing and real-time callback discipline

Original assignment and setup

Mix two generated tones with adjustable gains using a platform audio API. Pass: no clipping under the documented gain policy, bounded callback work and a recorded underrun or deadline-stress test.

Begin with offline samples and the native C++ harness. Then use one platform audio API already available on the learner's machine. Python allocation-heavy code is only a mathematical reference, not the real-time callback implementation.

Exercise kernel contract

Input sample_rate>0, frames 0..65536, 1..8 frequencies strictly between 0 and Nyquist, matching gains and optional phases in radians. Sum absolute gains <=1. Return mono samples and wrapped final phases. This offline Python kernel is not a real-time callback; native/device work remains in the full lab.

Starter: `starter/python/exercises/lesson_27_3.py`

Demonstration: `starter/python/demos/lesson_27_3.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 27.3`

Procedure

1. Calculate the callback interval for 256 frames at 48,000 frames/s: 5.333 ms. State channels, sample format and whether the API requests frames or interleaved scalar samples.
2. Generate two tones with phase carried across blocks. Compare a single long block with two consecutive shorter blocks. Apply an explicit sum-of-absolute-gains bound or another documented clipping policy.
3. Build audio_harness.cpp. Inspect its eight-voice cap, sixteen-command cap, prebuilt wavetable and SPSC ownership. Explain why a lock-free data structure alone does not prove bounded total work or a platform deadline.
4. Connect an actual device callback with all buffers/resources prepared before start. Move logging, loading and deletion to non-callback code; record command overflow and device loss in bounded telemetry.
5. Run a stress experiment under controlled background load. Record callback sizes, processing distribution and underrun/device error evidence available from the API. State precisely which deadline was measured. Keep volume low for the first audible run.

Adversarial and boundary cases

Zero voices; full command queue; invalid frequency/gain; callback block-size mismatch; device unplug/reopen; shutdown while commands are pending; a deliberate loading operation that must remain outside the callback.

Required evidence

Format sheet, gain proof, phase-continuity test, native build log, callback ownership diagram and device-level stress record with raw data.

Pass conditions

No clipping under the declared gain policy; work/storage are bounded; a real platform callback and stress test are recorded. Passing the offline harness alone does not finish this lab. Missing device access is recorded as deferred, not passed.

Native engine transfer

Connect the native mixer pattern to your engine audio adapter. Keep device-specific code outside the simulation and demonstrate graceful operation when no output device exists.

Extension after the required pass

Add an audio-thread-owned sample pool with generation handles and deferred reclamation. Test stop/restart without freeing data the callback can still read.

Lab 27.4: Spatial audio, animation events and multimodal accessibility

Original assignment and setup

Add spatial panning, separate music/effects controls and a visual equivalent for a critical audio cue. Pass: replay does not duplicate one-shot events and a muted-audio test still allows the required task.

Use one critical event, such as a hazard warning or goal activation. Give it audio and visual representations driven by the same semantic event. Keep music and effects categories independently adjustable.

Exercise kernel contract

Input pan in [-1,1], seen/events lists of [epoch_string,tick,entity,index], and min_tick>=0. Return gains [left,right], emitted unique events in arrival order, and retained identities sorted. Ignore events older than min_tick. Inputs at most 4096 identities each; retained union cap 8192. Epoch is part of identity. Full runtime must prune by its rollback horizon.

Starter: `starter/python/exercises/lesson_27_4.py`

Demonstration: `starter/python/demos/lesson_27_4.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 27.4`

Procedure

1. Derive equal-power pan gains cos(theta), sin(theta), with theta=(pan+1)*pi/4. Verify gain_L^2+gain_R^2=1 and distinguish constant signal power from guaranteed perceived loudness.
2. Assign each one-shot a stable identity (match_epoch, tick, entity_id, event_index). Keep presentation deduplication separate from rollback state so resimulation does not replay an already heard sound.
3. Implement a retained-event window. Define what happens to an event older than the window and to events from a new match epoch. Prune storage without allowing unbounded growth.
4. Add category controls, a captions/text or symbol cue, and a current-binding prompt. Trigger the event while animation and simulation advance at different rates.
5. Ask a tester to complete the critical task with audio muted and then with only one output channel. Record observations, not just whether the visual cue exists. Check that color alone is not required to distinguish the important state.

Adversarial and boundary cases

Duplicate/reordered event; rollback over an already presented event; new epoch reusing entity IDs; muted effects; extreme pan; event just outside retention; cue overlapping another UI element.

Required evidence

Power calculation, event identity and retention policy, duplicate-event log, settings persistence and a muted-task observation record.

Pass conditions

No duplicate one-shots within the stated supported replay window; both category controls work; the critical task remains understandable without audio under an observed test.

Native engine transfer

Route your engine events into interchangeable audio, visual and optional haptic adapters. Demonstrate one critical task with audio muted and no dependence on color alone.

Extension after the required pass

Add spatial attenuation and compare authored versus physically motivated curves. Keep the semantic critical cue available even when the source is attenuated or off screen.

Module 27 project

Build an Articulated Signal Station. A two-link arm or small character can idle, move and perform one interrupted action. Its motion must have a defined hierarchy, bind pose and blend policy. One semantic gameplay event drives an audio cue and a visual alternative. The example theme is optional; an original equivalent is welcome.

Prepare the native audio resources outside the callback, route bounded commands through an ownership-safe channel and keep music/effects controls independent. Implement the actual platform audio path and record a stress test. The supplied offline harness establishes only the behavior it executes; it does not establish the student’s device deadline or the absence of dropouts.

Run the simulation at a fixed step and show presentation at two rates. Roll back across an already presented event and demonstrate that a one-shot is not duplicated. Complete a muted-audio user task and record where the tester hesitated. Do not claim broad accessibility from a single participant.

Deliver transform/space definitions, action state table, IK tests, callback resource/work caps, raw device observations, event identity/retention policy, settings save/load and the muted-task record. The reviewer should be able to trigger cancellation and identify which system owns motion, sound emission and cleanup.

Module review evidence

Revisit the five review questions in the handbook after completing the integrating project. Use a changed scene or input to demonstrate transfer, not just a repeated definition. Attach your module project record before proceeding.

Module 28: Game AI, decision systems and machine learning

Lab 28.1: Navigation graphs, steering and dynamic obstacles

Original assignment and setup

Combine A* with a steering controller and a blocked-path detector. Pass: agent size is respected, unreachable goals terminate or report failure, and repeated replanning has a bounded budget.

Use a small grid with static walls, a moving obstacle and at least two agent sizes. A path is a route through a representation; steering and collision response still have to execute it.

Exercise kernel contract

Input rectangular grid strings ('.' free, '#' blocked), start/goal [x,y], budget>=0. Maximum 128 by 128. Return status found/unreachable/budget_exhausted, path and expanded count. Unit moves, Manhattan heuristic, lexicographic heap tie-break. Blocked start/goal is unreachable. Goal removal counts as an expansion.

Starter: `starter/python/exercises/lesson_28_1.py`

Demonstration: `starter/python/demos/lesson_28_1.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 28.1`

Procedure

1. Choose cell costs and a heuristic consistent with permitted moves. For unit-cost four-neighbor movement, justify Manhattan distance. Declare tie-breaking and the unit charged against the expansion budget.
2. Implement A* and distinguish found, unreachable and budget_exhausted. Verify path length against an independent breadth-first search on seeded unit-cost maps.
3. Inflate obstacles by agent footprint or otherwise encode clearance. Show one corridor that admits a small agent but not a large agent.
4. Follow the route with a steering controller and a blocked-progress detector based on actual advancement, not just elapsed time. Replan under a per-tick/per-agent work cap.
5. Attach a navigation-world generation to asynchronous requests. Reject an old result after the map changes; preserve a safe fallback such as stop-and-replan rather than following stale geometry.

Adversarial and boundary cases

Blocked start/goal; unreachable island; expansion budget exhausted; moving wall closes the next cell; target changes while search runs; stale map generation; oscillating replans.

Required evidence

Heuristic explanation, BFS comparison results, size-clearance scene, status traces, work-count distribution and a blocked-route recovery capture.

Pass conditions

Legal paths respect footprint; unreachable and exhausted cases are not conflated; following/replanning work is bounded; stale results cannot replace a newer request.

Native engine transfer

Add a globally budgeted navigation queue and stale-result rejection to the engine. Demonstrate a wide and a narrow actor receiving different valid routes.

Extension after the required pass

Compare per-agent A* with a shared flow field for many agents seeking one target. Include construction cost, invalidation and different agent sizes.

Lab 28.2: Behavior trees, utility systems and explainable decisions

Original assignment and setup

Implement either a small behavior tree or utility selector with a trace viewer. Pass: action cancellation releases resources and three induced failure cases produce understandable explanations.

Build one inspectable agent with patrol, investigate, flee and wait behavior. A utility selector or small behavior tree is acceptable. The kernel demonstrates a stable utility selector, not action execution lifecycle.

Exercise kernel contract

Input scores mapping nonempty action names to finite scores or null (invalid), current action or null, and nonnegative margin. Return action and reason. Highest score wins, lexicographic name breaks ties. Retain a valid current action unless alternative score is strictly greater than current+margin.

Starter: `starter/python/exercises/lesson_28_2.py`

Demonstration: `starter/python/demos/lesson_28_2.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 28.2`

Procedure

1. Define the data visible to a decision as an immutable tick snapshot. Make invalid/missing observations explicit rather than encoding them as a misleading zero score.
2. Implement selection with a switch margin and deterministic ties. Explain why hysteresis reduces thrashing but can delay a desirable switch.
3. Implement action start, tick and cancel. Allocate a navigation request or temporary resource on start; release it exactly once on completion or cancellation.
4. Write a trace containing chosen action, candidate scores/status, rejection reason and relevant observations. Show it beside the agent or in a readable log.
5. Inject three failures: missing target, unreachable route and interrupted action. Confirm the trace explains the decision and the recovery does not leak resources or leave two actions owning movement.

Adversarial and boundary cases

Equal scores; non-finite score; all actions invalid; action removed while current; score change smaller than margin; cancellation during an asynchronous reply.

Required evidence

Behavior diagram, score/condition definitions, three failure traces, cancellation counter and resource-lifetime tests.

Pass conditions

Every decision has an inspectable reason; invalid actions are not selected; action cancellation releases ownership exactly once; induced failures lead to a documented finite fallback.

Native engine transfer

Expose a bounded decision trace in the engine inspector, including facts, scores, chosen action, cancellation reason and outstanding request generation.

Extension after the required pass

Compare a behavior tree and utility selector on the same scenario set. Measure authoring/debugging differences as well as behavior outcomes.

Lab 28.3: Adversarial search, planning and simulation budgets

Original assignment and setup

Implement minimax with alpha-beta or a small MCTS agent for a tiny game. Pass: legal moves are always returned within a defined budget or fallback, and evaluation includes multiple seeds and opponents.

Choose a tiny legal game, such as tic-tac-toe or a reduced board game. The supplied alpha-beta kernel evaluates a tree; your lab must produce a legal action from real game states.

Exercise kernel contract

Input tree nested nonempty lists with finite numeric leaves, budget>=0 and maximizing boolean (default true). Return value, nodes and complete. Validation caps structure at 10000 nodes/depth 64 before search; node budget counts visited evaluation nodes including root. On budget exhaustion return value null rather than an incomplete value. Full lab adds legal actions, iterative deepening and fallback.

Starter: `starter/python/exercises/lesson_28_3.py`

Demonstration: `starter/python/demos/lesson_28_3.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 28.3`

Procedure

1. Define terminal utilities and legal-move generation. Test terminal states and states with exactly one legal move before adding search.
2. Use exhaustive minimax as an oracle on shallow trees. Implement alpha-beta with a deterministic move order and count visited nodes using the same convention in every comparison.
3. Add a strict budget and iterative deepening. Retain the best move from the last fully completed depth. When no depth completes, return a prevalidated legal fallback rather than an uninitialized action.
4. Compare at least three opponents or policies over a declared set of seeds/start states. Record win/loss/draw, illegal actions, nodes and elapsed decision time.
5. Change move ordering without changing the game result. Explain why node count can change dramatically even when the returned minimax value does not. Keep wall-clock and deterministic node budgets distinct.

Adversarial and boundary cases

Zero remaining decision budget; terminal board; forced loss; equal-value moves; budget interrupted halfway through a root move; invalid board; adversarially bad move order.

Required evidence

Game rules, legal-move tests, oracle comparison, fallback trace, opponent/seed manifest and decision-cost distribution.

Pass conditions

Every nonterminal decision returns a legal move under its declared work budget or explicit fallback; shallow search matches the oracle; evaluation spans multiple opponents/start conditions.

Native engine transfer

Integrate a decision job with a legal fallback and explicit completion status. Record how increasing the budget changes both response latency and decision quality.

Extension after the required pass

Implement MCTS and compare it with alpha-beta at equal measured runtime and at equal simulation count. Discuss why those comparisons answer different questions.

Lab 28.4: Learning systems, neural inference and responsible evaluation

Original assignment and setup

Train a small classifier or value estimator on synthetic game data and compare it with a rules baseline. Pass: independent splits, error analysis, inference timing, reproducible seeds and a fallback behavior are reported.

Use synthetic episode data for a bounded classifier or value estimator. The bundled logistic example needs no GPU or third-party machine-learning package. It does not establish performance on real players or the learner’s game.

Exercise kernel contract

Input matching finite feature x and weight w lists (1..64), bias b, label y exactly 0 or 1, and positive learning rate lr<=1. Return updated w,b and probability_before. Stable sigmoid avoids overflow. This kernel is one update, not a completed evaluation or trained policy.

Starter: `starter/python/exercises/lesson_28_4.py`

Demonstration: `starter/python/demos/lesson_28_4.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 28.4`

Procedure

1. Write a feature schema and label-generation rule. Identify which features are available at decision time; exclude future outcomes and information hidden from the agent.
2. Split by independent episode, scenario or seed before fitting preprocessing or selecting a model. Record exact split IDs and check disjointness. Do not randomly split adjacent frames from the same episode.
3. Derive one logistic gradient step, implement the kernel and verify it by finite differences. Train a small model and compare it with a rules/majority baseline selected using training information.
4. Choose model settings using validation only. Freeze the artifact and evaluate the held-out test once. Report confusion/error categories, not only accuracy; inspect failures under a shifted synthetic distribution.
5. Time frozen inference on the target machine and integrate a budgeted fallback for invalid features, a missing model or an expired decision. A good offline score does not justify an unbounded runtime dependency.

Adversarial and boundary cases

Leaked episode IDs; missing feature; NaN input; imbalanced labels; changed feature scale; model version mismatch; latency budget exceeded; shifted scenario distribution.

Required evidence

Dataset generator and seeds, episode split manifest, gradient check, baseline/model error table, frozen model metadata, inference samples and fallback tests.

Pass conditions

Independent splits are verified; the baseline comparison and error analysis are reproducible; target-machine inference timing and a tested fallback are supplied. Learning that a rules baseline is preferable is an acceptable result.

Native engine transfer

Export a versioned model and feature schema, then integrate a bounded inference adapter with a rules fallback. Keep model output advisory; authoritative simulation still validates legal actions.

Extension after the required pass

Reproduce the same model in a maintained ML framework and compare preprocessing, numeric outputs and deployment overhead. Pin the version and avoid conflating training hardware with inference hardware.

Module 28 project

Build an Inspectable Courier Agent. The agent travels to a target, responds to a blocked route and chooses among a small set of understandable actions. Give every decision a readable trace, including invalid alternatives and action cancellation. Use a strict search/replan budget and a safe behavior when it is exhausted.

Compare a rules baseline with one search-based or learned component. The lesson on learning still requires its synthetic training/evaluation exercise even when the integrated agent uses search. Select evaluation scenarios before looking at results: a clear route, a narrow corridor, a newly blocked passage, a missing target and at least one adversarial case.

Report completion/failure categories, time or work per decision, cancellation/resource behavior and reproducible seeds. For learned behavior, use independent episode splits and a frozen artifact. For search, provide legal-move/route checks and a completed-iteration fallback. A baseline outperforming the sophisticated component is useful engineering evidence.

Deliver a behavior specification, scenario manifest, raw traces, oracle comparisons, cost distributions and a short design decision explaining the chosen approach. A reviewer should cause a failure and explain the recovery from the trace without reading every source line.

Module review evidence

Revisit the five review questions in the handbook after completing the integrating project. Use a changed scene or input to demonstrate transfer, not just a repeated definition. Attach your module project record before proceeding.

Module 29: Multiplayer simulation and online services

Lab 29.1: Authority, prediction and reconciliation

Original assignment and setup

Implement a one-dimensional or grid movement server with client prediction. Pass: invalid movement is rejected and induced delay produces documented corrections without permanently diverging state.

Use a scalar or grid movement authority and two clients. The companion simulator supplies deterministic in-process transport faults. For the complete lab, put the same message contract behind a local socket or separate-process adapter.

Exercise kernel contract

Input authoritative integer position, ack>=0, pending list [sequence,delta]. Sequence is nonnegative; delta is integer -1,0,1. Position is clamped per input to [-100,100]. Deduplicate identical sequences, reject conflicting duplicates, replay only sequence>ack sorted. Return position and remaining pending pairs. Kernel assumes ack really denotes contiguous processed input.

Starter: `starter/python/exercises/lesson_29_1.py`

Demonstration: `starter/python/demos/lesson_29_1.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 29.1`

Procedure

1. Define input sequence, match epoch, controlled entity and legal movement bounds. The server receives intentions, not a trusted final client position.
2. Predict locally using the same movement step. Retain ordered unacknowledged commands. A server acknowledgement means every command through that sequence was applied, not merely received.
3. Buffer out-of-order inputs within a cap, ignore exact duplicates and reject conflicting duplicates. Advance the contiguous acknowledgement only when gaps are filled.
4. On a newer snapshot, restore authoritative state, discard acknowledged inputs and replay pending commands exactly once. Record correction size separately from render smoothing.
5. Run delay, loss, duplication and reorder scenarios for two clients, then stop generating input and drain reliably as a clearly labeled convergence test. Repeat with an unauthorized owner and invalid movement. Add the local transport/process adapter without changing authority rules.

Adversarial and boundary cases

Input 41 arriving before 40; duplicated input; stale snapshot; client claims another entity; sequence from an old match; oversized backlog; reconnect with pending commands.

Required evidence

Message schema, authority diagram, raw packet/correction traces, two-client fault results, invalid-input results and a separate-process/socket execution record.

Pass conditions

Authority rejects invalid ownership/movement; acknowledgement remains contiguous; clients converge after a documented drain; old snapshots do not overwrite newer state. In-process results alone do not demonstrate socket integration.

Native engine transfer

Create an engine command schema and separate authoritative, predicted and render-only state. Record one correction with the exact acknowledged and replayed input IDs.

Extension after the required pass

Add an authoritative obstacle that invalidates a predicted move. Compare simulation correction with visual smoothing and ensure smoothing does not change collision state.

Lab 29.2: Snapshots, interpolation, relevance and lag compensation

Original assignment and setup

Add remote interpolation and relevance filtering. Pass: out-of-order snapshots are handled, missing baselines trigger recovery, and the report explains both visual delay and hidden-state exposure policy.

Add a remote entity, timestamped snapshots and a relevance boundary. Keep full snapshots first; delta compression is a later step requiring a named baseline.

Exercise kernel contract

Input nonempty snapshots [server_time,position], time render target. Maximum 4096 samples. Sort timestamps, deduplicate identical values, reject conflicting duplicate times and non-finite values. Linear interpolation inside bracket; hold nearest endpoint outside. Return position only; full lab must track hold/delay/recovery telemetry.

Starter: `starter/python/exercises/lesson_29_2.py`

Demonstration: `starter/python/demos/lesson_29_2.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 29.2`

Procedure

1. Write the clock relationship and render target time. Interpolate between snapshots bracketing that time; define explicit behavior before the first or after the newest snapshot.
2. Accept reordering into a bounded time-sorted buffer while rejecting conflicting duplicates and snapshots from another epoch. Test timestamp values independent of packet arrival time.
3. Implement interest/relevance selection on the server before serialization. Inspect bytes or decoded payload to verify a hidden entity is absent, not merely invisible in the client renderer.
4. Add one delta message with a baseline identifier. Drop that baseline deliberately and request/resend a full snapshot; never apply deltas to an arbitrary recent state.
5. Measure the chosen interpolation delay and distinguish it from one-way network latency. Show a buffer underrun with bounded extrapolation or an intentional hold policy, then recovery.

Adversarial and boundary cases

Equal timestamp conflict; reversed arrival order; missing baseline; visibility transition; no bracketing newer sample; teleport discontinuity; epoch reset; hidden entity included in an otherwise filtered batch.

Required evidence

Timestamp/arrival trace, interpolation calculation, payload relevance audit, missing-baseline recovery and a delay/quality comparison.

Pass conditions

Reordering is handled under the buffer policy; missing baselines trigger recovery; hidden-state policy is enforced in transmitted data; visual delay is honestly accounted for.

Native engine transfer

Implement authority-side relevance filtering and capture the outgoing payload. Demonstrate that a hidden entity is absent from the unauthorized client, not merely invisible in the renderer.

Extension after the required pass

Compare fixed interpolation delay with an adaptive jitter buffer. Cap changes and record the additional delay rather than hiding it in a smoothness score.

Lab 29.3: Lockstep, rollback and deterministic replay

Original assignment and setup

Implement rollback or input lockstep for the tiny deterministic game. Pass: a delayed-input test converges, state hashes detect a deliberately introduced divergence, and audio or reward side effects are not duplicated.

Use a tiny deterministic simulation with an explicit state-before-tick convention. The kernel replays an integer tape; integrate full entity/RNG state and presentation events for the complete lab.

Exercise kernel contract

Input integer initial, inputs list of integer deltas -1,0,1, corrections list [tick,delta], and oldest retained pre-tick index. Maximum 10000 inputs. Build predicted history, restore earliest corrected S_tick, replay through end, return states including S_0. Reject corrections older than oldest or outside tape, and conflicting duplicates. This bounded scalar model does not snapshot a full physics engine.

Starter: `starter/python/exercises/lesson_29_3.py`

Demonstration: `starter/python/demos/lesson_29_3.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 29.3`

Procedure

1. Define S_t as state immediately before applying input I_t. Save S_t before each step, and define the maximum rollback window in ticks and bytes.
2. Predict a missing input, then deliver its correction late. Restore S_t, replace I_t and replay t through the last executed tick. Compare every corrected state against a fresh authoritative replay.
3. Add canonical hashes and inject one hidden-state mismatch. Locate its first divergent tick. Test a correction at the oldest retained tick and one just too old.
4. Generate an audio event and a reward during the corrected interval. Separate replayable simulation consequences from irreversible external effects. Deduplicate presentation and make authoritative rewards idempotent.
5. For lockstep as an alternative, replace rollback with an input barrier and document timeout/reconnect behavior. Do not combine both protocols without stating which owns each state transition.

Adversarial and boundary cases

Correction at tick zero; multiple corrected ticks; duplicate correction; correction outside history; RNG omitted from snapshots; deleted/reused entity ID; event heard before rollback.

Required evidence

Pre-tick convention sheet, history-memory calculation, delayed-input replay comparison, first-divergence trace and audio/reward duplication tests.

Pass conditions

Delayed inputs converge within the supported window; intentional divergence is detected; unsupported old inputs have an explicit recovery path; external side effects are not duplicated.

Native engine transfer

Add a bounded rollback ring to your engine or a separate deterministic two-client demonstration. Keep irreversible side effects behind an explicit confirmation/idempotency boundary.

Extension after the required pass

Measure worst-case resimulation time at the maximum window and compare it with the frame budget. A low average rollback cost does not establish a safe maximum.

Lab 29.4: Match services, identity, persistence and failure recovery

Original assignment and setup

Build a local match registry and idempotent result store. Pass: duplicate submissions do not duplicate rewards, unauthorized edits fail, and simulated service loss produces a documented client outcome.

Use a local match registry plus an idempotent result store. The SQLite companion demonstrates result transactions and fresh-process persistence; the registry and real identity adapter remain learner work.

Exercise kernel contract

Input principal must equal match_authority; operation has exactly id/player/points with nonempty bounded strings and integer points 0..1000. ledger is a list of previous operations; balances is a mapping to nonnegative integer points. Return copied ledger/balances and applied boolean. Exact retries have no effect; same ID/different payload is a conflict. Synthetic principal is NOT authentication. Full lab uses a database transaction and process restart.

Starter: `starter/python/exercises/lesson_29_4.py`

Demonstration: `starter/python/demos/lesson_29_4.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 29.4`

Procedure

1. Define registry states waiting, active, completed and expired, plus permitted transitions. Give every match a unique epoch and reject results for an unknown or unowned match.
2. Implement one atomic transaction that checks the result ID/payload, records the operation and applies the reward. Exact retries must be no-ops; the same ID with a different payload must conflict.
3. Inject failure before commit and after commit but before response delivery. Retry each request and verify that the balance is changed exactly once only when the transaction committed.
4. Restart the service process and query persisted state. Simulate service loss during a match and define the client result: pending submission, retry status or another explicit recoverable outcome.
5. Add an identity boundary that maps a verified session to a principal. In the local teaching model, label identities synthetic and test ownership separately. Never present a trusted input string as production authentication.

Adversarial and boundary cases

Duplicate result; same ID/different reward; client edits another match; expired match; process restart; two concurrent retries; database unavailable; result arrives before match completion.

Required evidence

Registry transition table, transaction trace, injected-failure results, restart log, ownership tests and client recovery-state capture.

Pass conditions

Retries do not duplicate rewards; conflicts and unauthorized edits fail; registry state is valid; restart retains committed results; service loss produces a documented client outcome.

Native engine transfer

Implement a file-backed local result store and prove exact retry behavior across a process restart. Add a match-state transition table and an unavailable-service client state.

Extension after the required pass

Add a transactionally written outbox for external notifications, then simulate delivery retries. Explain why a database transaction cannot atomically include an arbitrary remote email/payment service.

Module 29 project

Build a Local Two-Client Simulation. Use one authoritative process and two local clients, or an equivalent process/socket layout with explicit identities and controlled inputs. The package’s in-process simulator is a reference for message semantics, not a substitute for the student’s transport adapter. No public endpoint or paid service is required.

Exercise delay, jitter, loss, duplication and reordering with seeded schedules. Include local prediction/reconciliation, a remote interpolation buffer and one missing-baseline recovery. Show which state is omitted by relevance filtering. Use rollback or input lockstep for a tiny deterministic mode, with a documented history/barrier policy and side-effect handling.

Add a local registry and transactional result store. Inject failure before and after commit and restart the service. Keep synthetic test identities clearly distinct from production authentication. Demonstrate an explicit client state while the service is unavailable rather than silently losing results.

Deliver protocol schemas, process topology, fault schedules, correction/replay traces, hidden-state payload audit, transaction/restart logs and limitations. Review asks what is authoritative, what is predicted, what can be replayed and what must be idempotent.

Module review evidence

Revisit the five review questions in the handbook after completing the integrating project. Use a changed scene or input to demonstrate transfer, not just a repeated definition. Attach your module project record before proceeding.

Module 30: Production quality and the undergraduate engine capstone

Lab 30.1: Profiling, frame budgets and performance regression tests

Original assignment and setup

Profile the capstone scene, optimize one verified bottleneck and add a regression benchmark. Pass: repeated before/after results, unchanged correctness and disclosed measurement uncertainty are included.

Choose one reproducible scene in the learner’s engine, a declared machine and a budget. The statistics kernel computes summaries from supplied numbers; its sample arrays are synthetic, not measured capstone performance.

Exercise kernel contract

Input nonempty finite positive times_ms list (max 100000) and positive budget_ms. Return count, mean_ms, p50_ms,p95_ms,p99_ms,max_ms,over_budget. Percentiles use ceil(p*N)-1. These input samples are not automatically real measurements; record their provenance separately.

Starter: `starter/python/exercises/lesson_30_1.py`

Demonstration: `starter/python/demos/lesson_30_1.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 30.1`

Procedure

1. Freeze build mode, scene seed, resolution, entity count and warm-up policy. State exactly where CPU, GPU or software-renderer timings start and stop.
2. Collect multiple independent runs with raw per-frame samples and a peak-memory method. Compute nearest-rank percentiles and over-budget counts. Keep loading and steady-state intervals labeled.
3. Use a profiler to identify one actual bottleneck. Form a hypothesis and a correctness-preserving change before measuring the outcome.
4. Alternate baseline and changed builds where practical. Report per-run distributions and run-to-run variation; keep correctness/replay/image checks unchanged.
5. Add a regression test using a defensible threshold or trend policy. A noisy shared runner may be suitable for correctness but unsuitable for a hard performance gate.

Adversarial and boundary cases

One 100 ms stall hidden by a good mean; timer unit error; debug versus release mismatch; insufficient warm-up; thermal/background-load change; a faster build with changed outputs.

Required evidence

Benchmark manifest, raw samples, profiler capture, hypothesis, before/after distributions, correctness comparison and regression policy.

Pass conditions

At least one measured bottleneck is investigated; repeated results and uncertainty are disclosed; output correctness remains valid. A negative or inconclusive optimization is acceptable when evidence supports it.

Native engine transfer

Add a reproducible capstone benchmark with raw frame durations, work counts and environment metadata. Keep measurement code out of the authoritative replay hash.

Extension after the required pass

Use Amdahl’s law to predict the maximum application benefit before optimizing a subsystem, then compare the predicted ceiling with measured improvement.

Lab 30.2: Compatibility, accessibility, localization and human testing

Original assignment and setup

Test the game at two window sizes, with remapped input and an expanded pseudo-localized text set. Pass: no critical clipped controls, a muted-audio task remains solvable and observed usability issues are tracked.

Use the capstone’s actual interface and controls. Automated text/binding tests supplement but do not replace human observation or platform testing.

Exercise kernel contract

Input text (max 4096 characters), bindings mapping unique action names to unique nonempty key labels, and action. Double literal vowels, preserve {identifier} placeholders, bracket the text. Reject malformed braces, absent action or conflicting key labels. Return text and prompt derived from current binding. This is an intentionally narrow localization model.

Starter: `starter/python/exercises/lesson_30_2.py`

Demonstration: `starter/python/demos/lesson_30_2.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 30.2`

Procedure

1. Create a compatibility matrix listing two window sizes, keyboard/controller states, audio on/off and the supported platform/build. Mark untested entries pending instead of guessing.
2. Remap one critical action, save settings and reload. Verify gameplay dispatch and every prompt use the current binding. Test focus changes and conflicting bindings.
3. Pseudo-localize visible strings with length expansion while preserving placeholders. Test plural, number and multiline cases relevant to the game; pseudo-localization is not a translation or a complete locale implementation.
4. At both window sizes, inspect critical controls, readable text, focus order and contrast. Test without a controller, then disconnect it during a loop if controller support is advertised.
5. Observe a tester completing the main loop with audio muted and a critical cue represented visually. Record the task, observed difficulty, intervention and fix. Do not claim broad accessibility from one checklist pass.

Adversarial and boundary cases

Expanded labels; missing glyph; wrong placeholder; binding changed while a prompt is visible; smallest window; audio muted; controller absent; UI focus stealing movement input.

Required evidence

Compatibility matrix, before/after binding record, pseudo-localized captures, usability observation and tracked issues with severity.

Pass conditions

No critical clipped control in tested configurations; remapping and persistence work; the critical muted task is solvable; findings are documented with observed evidence and limits.

Native engine transfer

Centralize action bindings and text resources in the engine. Have an independent tester complete the loop with a changed binding and muted audio; record and retest actual issues.

Extension after the required pass

Test with players who use different input/access needs, with consent and respectful task framing. Treat their feedback as individual evidence rather than a universal population result.

Lab 30.3: Packaging, patching, modding and supply-chain boundaries

Original assignment and setup

Package the game and implement a data-only mod with a versioned schema. Pass: a clean-folder launch works, oversized or malformed mods are rejected and an interrupted-update simulation preserves a valid version.

Package the learner’s game with a versioned data-only mod schema. No arbitrary script execution is needed. The mod kernel validates texture metadata only; actual loading, byte caps and filesystem handling are additional boundaries.

Exercise kernel contract

Input is the mod object itself: exactly version=1, title (1..80 chars), textures (<=16) each exactly name,width,height. Name is unique [A-Za-z0-9_-]{1,64}; integer dimensions 1..2048; RGBA8; total <=16 MiB. Return texture_count,total_bytes. Reject unknown fields, booleans as dimensions, unsupported versions and excessive resources before allocating texture payloads.

Starter: `starter/python/exercises/lesson_30_3.py`

Demonstration: `starter/python/demos/lesson_30_3.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 30.3`

Procedure

1. Write the schema with allowed keys, versions, length/count limits and resource-size limits. Validate metadata before allocation, then enforce an independent byte cap while reading payloads.
2. Use safe asset identifiers rather than trusting archive paths. Reject traversal, absolute paths, duplicate names, unknown fields and unsupported versions. State whether symbolic links and compressed archives are allowed.
3. Stage a complete new release separately from the active release, validate it, then switch a small activation pointer using a documented platform operation. Keep a known-valid old release until activation succeeds.
4. Inject interruption before save replacement and before update activation. Verify an old or new valid state survives rather than a partially written active state. State power-loss durability separately from process-level interruption tests.
5. Extract into a clean folder and launch using only documented commands/files. Record asset provenance and third-party licenses. A hash detects a mismatch against an expected value but does not by itself authenticate the publisher.

Adversarial and boundary cases

Unsupported schema; unknown key; boolean dimension; integer overflow in a native implementation; resource bomb; path traversal; interrupted save; corrupted staging asset; malicious replacement of both data and its unsigned hash.

Required evidence

Schema, malicious/oversized fixture set, update state diagram, interruption tests, provenance list and independent clean-folder launch record.

Pass conditions

Clean launch succeeds; malformed or oversized data is rejected before dangerous allocation/use; interrupted update leaves a valid version. Scope and authenticity/durability limits are explicit.

Native engine transfer

Create a versioned data-only mod schema and an actual release folder. Test rejected input, interrupted staging and a save that remains intact when installation files are replaced.

Extension after the required pass

Add a verified signed manifest using an established platform/library, then document key rotation and rollback policy. Do not invent a signing scheme.

Lab 30.4: Capstone review, release and technical postmortem

Original assignment and setup

Complete Gate F and the oral questions in the handbook. Pass: another tester finishes a full loop, the release is reproducible, critical defects are resolved or clearly blocked from release, and the postmortem cites measured evidence.

Complete Gate F using the game on your existing engine. The package supplies specifications and review tools, not a finished student game or a preapproved assessment.

Exercise kernel contract

Input scores with exactly gameplay,systems,correctness,production,performance,documentation,oral; maxima 20,20,20,15,10,10,5. checks with exactly clean_launch,save_recovery,critical_defects_resolved,independent_loop,live_change,network_demo (boolean). Return score,passed,blockers. Pass requires score>=80 and all checks true. This evaluates supplied claims only; it cannot verify human review or approve a capstone automatically.

Starter: `starter/python/exercises/lesson_30_4.py`

Demonstration: `starter/python/demos/lesson_30_4.py`

Test: `python starter/python/run_tests.py --target exercises --lesson 30.4`

Procedure

1. Freeze a release candidate with one complete original loop, instructions, settings, save/load, diagnostics, collision, articulated/animated motion, bounded audio and inspectable AI.
2. Assemble the evidence ledger. Separate automated demonstrations supplied with this package from results obtained on your own engine and machine. Link every acceptance claim to an artifact.
3. Run the declared compatibility and failure matrix, including interrupted saves, absent devices, exit during loading and malformed mods when exposed. Resolve release blockers or remove the unsafe feature from the supported release.
4. Have another tester follow the clean launch instructions, finish the loop and reproduce a bug regression. During review, make a small live change and rerun the relevant test.
5. Write the postmortem: expected behavior, measured outcome, most important incorrect assumption, one rejected optimization, remaining limitations and the next bounded improvement. Answer the oral prompts without relying on memorized code.

Adversarial and boundary cases

Package works only on the author’s machine; missing licensed asset; hidden build dependency; critical defect labeled merely cosmetic; passing score with a failed mandatory gate; claim without reproducible evidence.

Required evidence

Release archive/source, build manifest, raw performance and failure logs, test matrix, review record, live-change evidence and a measured technical postmortem.

Pass conditions

All Gate F mandatory evidence is present, no release blocker remains, the reviewer finishes the loop, and the rubric reaches the documented threshold. A script cannot certify human review or override a blocker.

Native engine transfer

Archive the reviewed engine/game revision with a clean-build record, an independently completed loop, one reproduced regression and a live-change review. Do not mark human review complete in advance.

Extension after the required pass

Conduct a graduate-style design review of one subsystem’s tradeoffs and limitations. Keep that analysis separate from claims that the undergraduate game itself is research novelty.

Module 30 project

Complete Gate F: the undergraduate software capstone. Integrate one original playable loop with the engine built in Volume 5. This is the culminating use of the module projects, not an additional unrelated large game. Reuse well-understood systems and keep content deliberately small.

Follow CAPSTONE_SPEC for scope, milestone evidence, test matrix and review. The multiplayer demonstration may remain separate from the main game. The game itself still requires collision/physics, animation or articulated motion, bounded audio, inspectable AI, settings, save/load, diagnostics and a tested distribution. Lesson 30.3’s data-only mod lab remains required even when the released game does not expose mods.

Collect repeated performance measurements on a declared machine, complete the compatibility/accessibility tasks, test failure recovery and arrange an independent loop completion and live change. A software-renderer route is acceptable where carried forward from Volume 5; do not claim GPU validation without a tested GPU backend.

Deliver the release, source/build instructions, provenance, raw evidence, reviewer record and postmortem. Grade the quality of reasoning and evidence, not art budget, map size or commercial success.

Module review evidence

Revisit the five review questions in the handbook after completing the integrating project. Use a changed scene or input to demonstrate transfer, not just a repeated definition. Attach your module project record before proceeding.

Gate F handoff

The five projects feed CAPSTONE_SPEC. Use the actual engine evidence, not copied demonstration outputs. Complete the compatibility matrix, device/transport work, failure recovery, raw performance record and independent review. A supplied template is empty until an actual experiment or observation is recorded.